

FINAL REVIEW

Encrypted Secret File Transfer Distributed Backend Database System

P. Sreshta	2520030217
V. Sree Pragnya	2520030238
G. Eesha Yuktha	2520030436

Under the Guidance
of

Dr. Y Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



KONERU LAKSHMAIAH EDUCATION FOUNDATION

GROUP NUMBER : 11

Name of Candidate	Role
G. Eesha Yuktha	Team Lead / Backend Database Engineer
V. Sree Pragnya	AI / Feature Development and Quality Assurance Engineer
P. Sreshta	Frontend Developer / UI-UX Engineer

ABSTRACT

The Encrypted Secret File Transfer Distributed Backend Database System is a secure file-sharing system designed to protect confidential files during storage and transfer. The system uses encryption to ensure that only authorized users can access shared files. A distributed backend manages file-related information across multiple components, while a database stores user details, file metadata, access permissions, and transfer records.

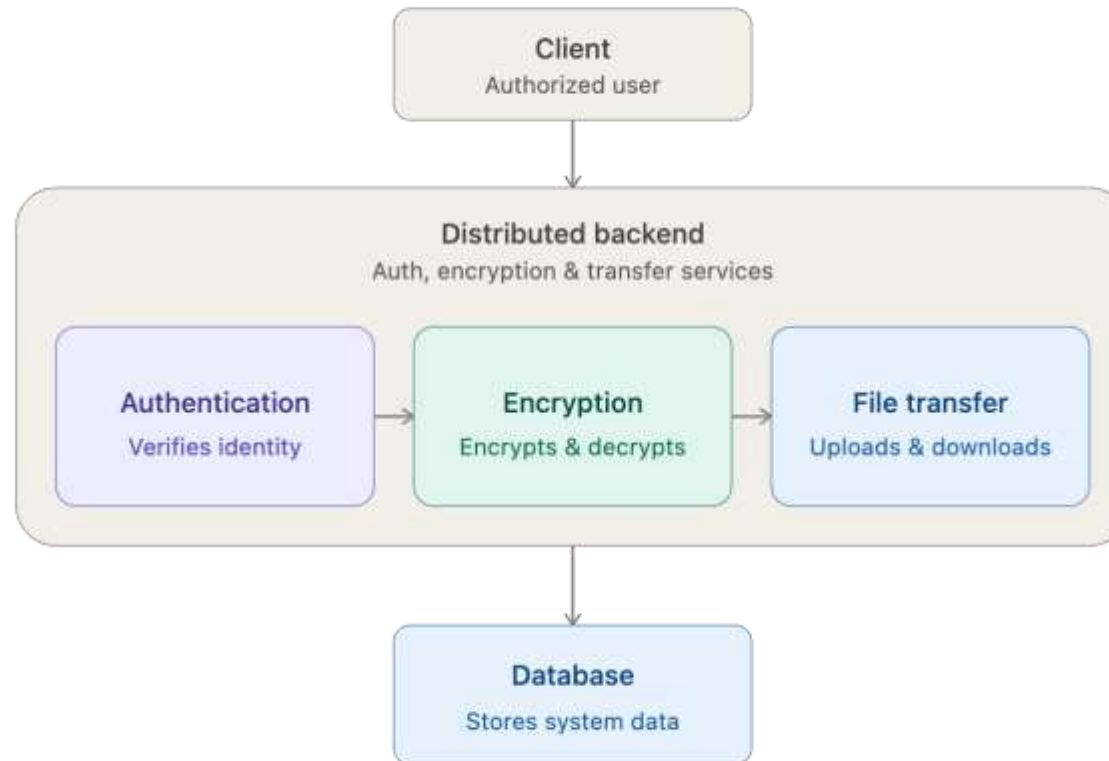
The project demonstrates how database systems and distributed backend technologies can work together to provide secure, reliable, and organized file transfer. Authentication and authorization mechanisms help prevent unauthorized access, while encryption protects files from being exposed during transmission. The project is suitable for understanding fundamental concepts of database management, backend development, distributed systems, and cybersecurity in a practical way.

INTRODUCTION

The Encrypted Secret File Transfer Distributed Backend Database System is developed to provide a secure method for transferring confidential files between authorized users. In a normal file-transfer system, sensitive files may be exposed if they are intercepted or accessed by unauthorized users. This project addresses this issue by encrypting files before transmission and allowing access only to authenticated users. The system maintains important information such as user accounts, file details, permissions, and transfer history in a database.

The project combines database systems with distributed backend development to create a reliable file-transfer application. The backend handles user authentication, encryption-related operations, file uploading and downloading, and communication between different system components. The database provides organized storage and retrieval of information, while the distributed architecture allows different backend services to perform specific tasks. This project helps students understand practical applications of databases, backend APIs, security, and distributed systems.

EXISTING SYSTEM



Purple = authentication, teal = encryption, blue = data handling, gray = client/backend

PROBLEM

Traditional file-transfer methods may expose confidential files to unauthorized users during transmission or storage. There is a need for a simple and secure system that encrypts files, authenticates users, manages access permissions, and stores file-transfer details in a database. The proposed system addresses these issues using an encrypted file-transfer mechanism with a distributed backend and database.

SOLUTION (Contribution)

- The proposed system provides a secure platform for transferring secret files between authorized users. Every file is encrypted on the server with AES-256-GCM before anything is written to disk, so only ciphertext is stored and unauthorized users cannot read it.
- A MongoDB database maintains user accounts, file metadata, permissions, and transfer logs. The distributed backend separates authentication, file handling and encryption, permission management, and audit logging into independent services behind an API gateway, giving a secure, manageable and fault-tolerant file-transfer system.

Objectives

- To develop a secure file-transfer system for confidential files.
- To implement user authentication and authorization.
- To encrypt files before storage or transfer.
- To maintain user and file information using a database.
- To control file access based on user permissions.
- To maintain records of file-transfer activities.
- To demonstrate distributed backend architecture.
- To provide a simple and organized interface for secure file sharing.

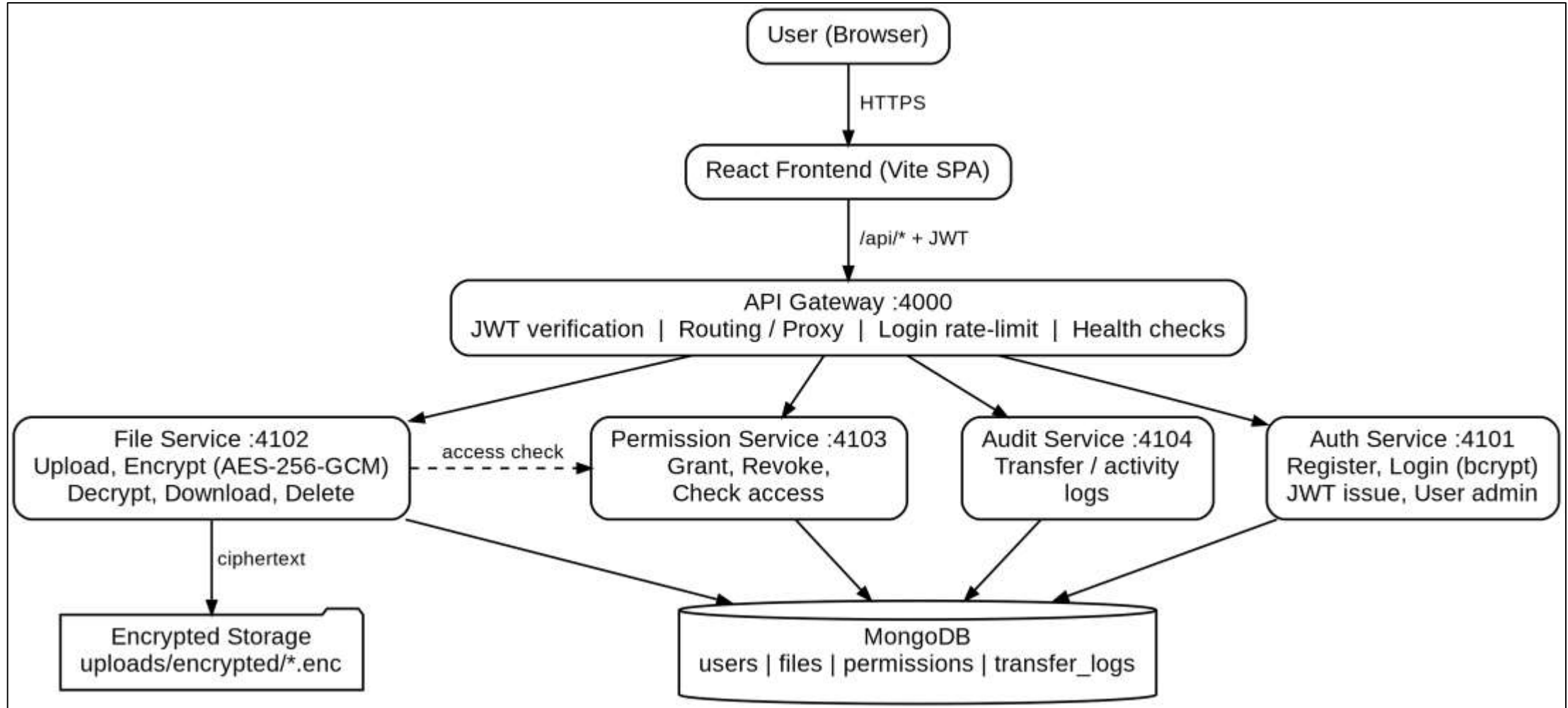
Modules

- **User Authentication Module** – Manages user registration, login, and authorization.
- **File Upload Module** – Allows users to upload files securely.
- **Encryption Module** – Encrypts files before storage or transfer.
- **File Transfer Module** – Shares encrypted files with registered users by email (grant and revoke).
- **Database Management Module** – Stores users, files, permissions, and transfer logs in MongoDB.
- **Access Control Module** – Checks owner or active permission before any file is read or decrypted.
- **File Download Module** – Allows authorized users to download and decrypt files.
- **Audit Log Module** – Records file uploads, downloads, transfers, and user activities.
- **Admin Module** – Lets administrators manage users and view system-wide logs and statistics.

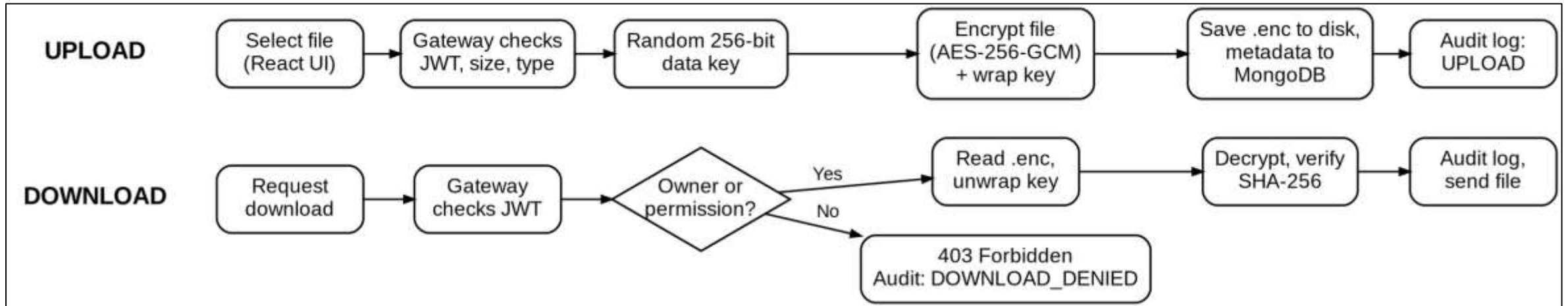
SOFTWARE REQUIREMENTS

- **Operating System** – Windows 10/11
- **Frontend** – React 18, Vite, HTML, CSS, JavaScript
- **Backend** – Node.js 18+ with Express (API gateway and microservices)
- **Database** – MongoDB with Mongoose
- **Encryption** – Node.js crypto (AES-256-GCM), bcrypt, JWT
- **API Testing** – Postman
- **Code Editor** – Visual Studio Code
- **Database Tool** – MongoDB Compass
- **Web Browser** – Google Chrome
- **Version Control** – Git/GitHub

SYSTEM ARCHITECTURE



WORKFLOW: UPLOAD AND DOWNLOAD



- Files are encrypted on the server (AES-256-GCM) before anything is written to disk; only ciphertext is stored.
- A fresh data key is generated for every file and wrapped with the master key; the wrapped key, IV and tags are kept in MongoDB.
- Permission is checked before any file is read or decrypted; every action, including denied attempts, is written to the audit log.

DATABASE DESIGN (MongoDB)

- **users (Auth Service)** – name, email (unique), passwordHash (bcrypt), role, active, lastLoginAt.
- **files (File Service)** – originalName, storedName, ownerId, size, sha256, encryption (iv, authTag, wrappedKey).
- **permissions (Permission Service)** – fileId, userId, level, status (active / revoked); unique on (fileId, userId).
- **transfer_logs (Audit Service)** – userId, fileId, action, status, metadata, timestamp.
- **Relationships** – one user owns many files; one file has many permissions; users and files have many logs.
- **Design choice** – each service touches only its own collection; cross-service data goes through APIs.

BACKEND CODE: ENCRYPTION

services/file-service/utils/crypto.js (encrypt)

```
const ALGORITHM = 'aes-256-gcm';

function gcmEncrypt(key, plaintext) {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
  const data = Buffer.concat([cipher.update(plaintext), cipher.final()]);
  return { iv, data, tag: cipher.getAuthTag() };
}

// Envelope encryption: one random data key per file
function encryptBuffer(plaintext) {
  const dek = crypto.randomBytes(32);
  const file = gcmEncrypt(dek, plaintext);
  const wrapped = gcmEncrypt(getMasterKey(), dek);
  dek.fill(0);
  return {
    ciphertext: file.data,
    meta: {
      algorithm: 'aes-256-gcm',
      iv: file.iv.toString('base64'),
      authTag: file.tag.toString('base64'),
      wrappedKey: wrapped.data.toString('base64'),
      wrapIv: wrapped.iv.toString('base64'),
      wrapTag: wrapped.tag.toString('base64')
    }
  };
}
```

services/file-service/utils/crypto.js (key and decrypt)

```
function getMasterKey() {
  const raw = process.env.FILE_ENCRYPTION_KEY;
  if (!raw) throw new Error('FILE_ENCRYPTION_KEY is not set');
  return /^[0-9a-fA-F]{64}$/.test(raw)
    ? Buffer.from(raw, 'hex')
    : crypto.scryptSync(raw, 'efts-master-key-salt-v1', 32);
}

function gcmDecrypt(key, iv, tag, data) {
  const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
  decipher.setAuthTag(tag);
  // final() throws if the data was tampered with
  return Buffer.concat([decipher.update(data), decipher.final()]);
}

function decryptBuffer(ciphertext, meta) {
  const b = (s) => Buffer.from(s, 'base64');
  const dek = gcmDecrypt(getMasterKey(), b(meta.wrapIv),
    b(meta.wrapTag), b(meta.wrappedKey));

  try {
    return gcmDecrypt(dek, b(meta.iv), b(meta.authTag),
      ciphertext);
  } finally {
    dek.fill(0);
  }
}
```

Each file gets its own AES-256-GCM key; the master key never leaves the server environment. GCM tags detect tampering.

BACKEND CODE: SECURE DOWNLOAD

services/file-service/controllers/fileController.js

```
// Returns 'owner' | 'download' | 'view' | null
async function accessLevel(file, user) {
  if (String(file.ownerId) === user.id) return 'owner';
  return perm.sharedLevel(String(file._id), user.id);
}

const download = asyncHandler(async (req, res) => {
  const file = await loadFile(req.params.id);
  const level = await accessLevel(file, req.user);
  if (level !== 'owner' && level !== 'download') {
    await fileLog(req.user, file, 'DOWNLOAD_DENIED', 'denied');
    throw new HttpError(403,
      'You do not have permission to access this file');
  }
  const encrypted = await storage.readEncrypted(file.storedName);
  const plaintext = decryptBuffer(encrypted, file.encryption);
  if (sha256(plaintext) !== file.sha256)
    throw new Error('integrity mismatch');
  await fileLog(req.user, file, 'DOWNLOAD', 'success');
  res.set({ 'Content-Disposition': 'attachment',
    'Cache-Control': 'no-store' });
  res.end(plaintext);
});
```

permission-service/routes/internal.js and shared/auth.js

```
// Only services holding INTERNAL_API_KEY may call /internal
function internalOnly(req, res, next) {
  const given = Buffer.from(
    String(req.headers['x-internal-key'] || ''));
  const expected = Buffer.from(
    String(process.env.INTERNAL_API_KEY || ''));
  if (given.length === expected.length && given.length > 0
    && crypto.timingSafeEqual(given, expected)) return next();
  next(new HttpError(403, 'Forbidden'));
}

// Is userId allowed to access fileId?
router.use(internalOnly);
router.get('/check', asyncHandler(async (req, res) => {
  const { fileId, userId } = req.query;
  const p = await Permission.findOne(
    { fileId, userId, status: 'active' });
  res.json({ allowed: !!p, level: p ? p.level : null });
}));
```

Authorization runs on the server before any file is read or decrypted; denied attempts are logged. Service-to-service calls fail closed.

BACKEND CODE: AUTHENTICATION

services/auth-service/controllers/authController.js (login)

```
const BCrypt_Rounds = 10;
// Compared when the email is unknown: similar response time
const DUMMY_HASH = bcrypt.hashSync('not-a-real-password-1',
    BCrypt_Rounds);

const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body || {};
  const user = await User.findOne(
    { email: email.trim().toLowerCase() }
  ).select('+passwordHash');
  const ok = await bcrypt.compare(password,
    user ? user.passwordHash : DUMMY_HASH);
  if (!user || !ok) {
    await logEvent({ action: 'LOGIN_FAILED', status: 'denied' });
    // same message for wrong email and wrong password
    throw new HttpError(401, 'Invalid email or password');
  }
  user.lastLoginAt = new Date();
  await user.save();
  await logEvent({ userId: user.id, action: 'LOGIN',
    status: 'success' });
  res.json({ token: signToken(user), user });
});
```

shared/auth.js (JWT used by every service)

```
function signToken(user) {
  return jwt.sign(
    { sub: String(user._id || user.id), email: user.email,
      name: user.name, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '8h',
      issuer: 'efts-auth-service' }
  );
}

// Every service re-verifies the JWT itself
function authenticate(req, res, next) {
  const token = extractBearer(req);
  if (!token)
    return next(new HttpError(401, 'Authentication required'));
  try {
    const p = verifyToken(token);
    req.user = { id: p.sub, email: p.email,
      name: p.name, role: p.role };
    next();
  } catch (e) {
    next(new HttpError(401, 'Invalid token'));
  }
}
```

Passwords are stored only as bcrypt hashes; JWTs are signed, issuer-checked, and expire after 8 hours.

FRONTEND CODE: API SERVICE

frontend/src/services/api.js (request helper)

```
const TOKEN_KEY = 'efts_token';
export const getToken = () => localStorage.getItem(TOKEN_KEY);

async function request(path, { method = 'GET', body } = {}) {
  const headers = {};
  const token = getToken();
  if (token) headers.Authorization = `Bearer ${token}`;
  if (body) headers['Content-Type'] = 'application/json';
  const res = await fetch(`/api${path}`, {
    method, headers,
    body: body ? JSON.stringify(body) : undefined
  });
  const data = await res.json().catch(() => null);
  if (res.status === 401 && token) onUnauthorized();
  if (!res.ok) throw apiError(res.status, data);
  return data;
}

export const api = {
  login: (email, password) => request('/auth/login',
    { method: 'POST', body: { email, password } }),
  files: (scope = 'all') => request(`/files?scope=${scope}`),
  share: (fileId, email, level = 'download') =>
    request('/permissions',
      { method: 'POST', body: { fileId, email, level } }),
};
```

frontend/src/services/api.js (upload and download)

```
upload(file, onProgress) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open('POST', '/api/files/upload');
    xhr.setRequestHeader('Authorization',
      `Bearer ${getToken()}`);
    xhr.upload.onprogress = (e) => e.lengthComputable &&
      onProgress(Math.round((e.loaded / e.total) * 100));
    xhr.onload = () => { /* parse JSON, resolve or reject */ };
    const form = new FormData();
    form.append('file', file);
    xhr.send(form);
  });
},

// Server authorises + decrypts; browser gets original bytes
async download(file) {
  const res = await fetch(`/api/files/${file.id}/download`,
    { headers: { Authorization: `Bearer ${getToken()}` } });
  if (!res.ok) throw apiError(res.status, await res.json());
  const url = URL.createObjectURL(await res.blob());
  const a = document.createElement('a');
  a.href = url; a.download = file.originalName;
  document.body.appendChild(a); a.click(); a.remove();
}
```

The browser only talks to the gateway under /api and never sees any encryption key.

FRONTEND CODE: UPLOAD AND SHARE

frontend/src/pages/UploadPage.jsx

```
export default function UploadPage() {
  const [file, setFile] = useState(null);
  const [progress, setProgress] = useState(null);
  const [error, setError] = useState('');

  function pick(f) {
    if (f && f.size > MAX_MB * 1024 * 1024) {
      setError(`File is larger than ${MAX_MB} MB`); return;
    }
    setFile(f || null);
  }

  async function submit() {
    setProgress(0);
    try {
      const r = await api.upload(file, setProgress);
      toast.success(`${r.file.originalName} encrypted and stored`);
      navigate(`/files/${r.file.id}`);
    } catch (e) { setError(e.message); setProgress(null); }
  }

  // ...
  <button className="btn btn-primary"
    disabled={!file || uploading} onClick={submit}>
    Encrypt & upload
  </button>
}
```

frontend/src/components/ShareModal.jsx and App.jsx

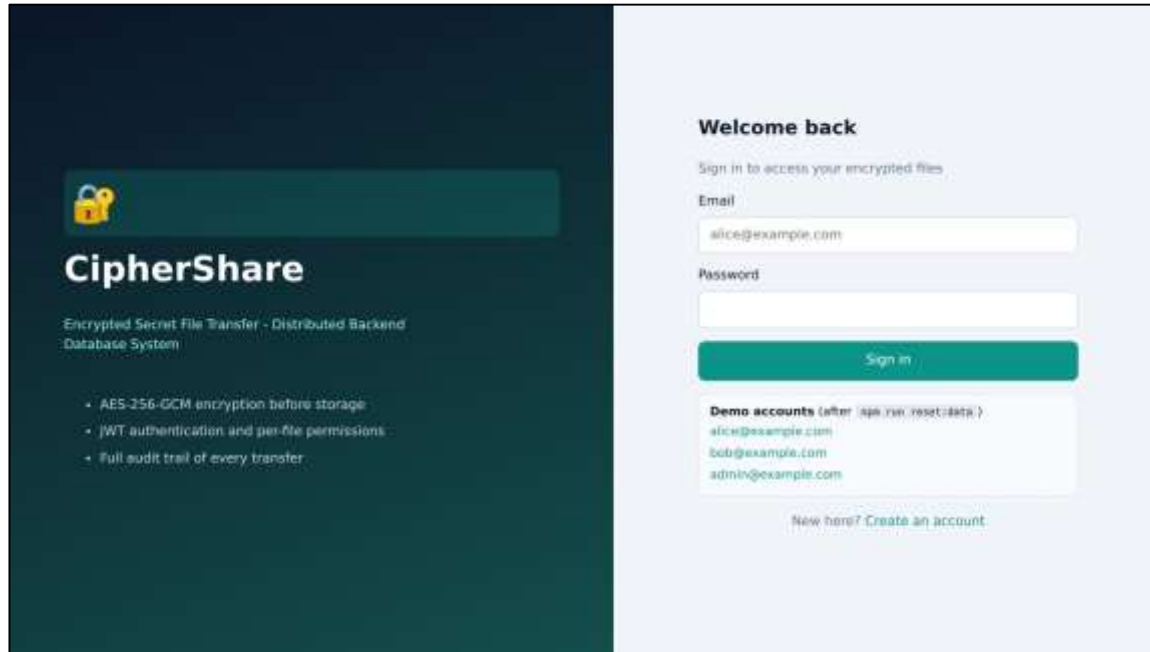
```
// ShareModal.jsx
async function submit(e) {
  e.preventDefault();
  setBusy(true); setError('');
  try {
    await api.share(file.id, email.trim());
    toast.success(`Shared "${file.originalName}"`);
    onShared && onShared();
    onClose();
  } catch (err) { setError(err.message); }
  finally { setBusy(false); }
}

// App.jsx - hash routing, pages only after login
const { user, loading } = useAuth();
const path = useHashRoute();
if (loading) return <Spinner label="Restoring session..." />;
if (!user) return path === '/register'
  ? <RegisterPage /> : <LoginPage />;
if (path === '/upload') page = <UploadPage />;
else if (path === '/shared') page = <FilesPage scope="shared" />;
else if (path === '/admin') page = <AdminPage />;
```

React pages call the API service; the share form sends the recipient email and the server resolves the user.

OUTPUT: LOGIN AND DASHBOARD

Login page



The login page features a dark teal background on the left with the CipherShare logo and a list of features: AES-256-GCM encryption before storage, JWT authentication and per-file permissions, and a full audit trail of every transfer. On the right, a light blue box contains the login form. It includes a 'Welcome back' message, a sign-in prompt, email and password input fields, a 'Sign in' button, and a section for demo accounts (alice@example.com, bob@example.com, admin@example.com) with a 'New here? Create an account' link.

Welcome back

Sign in to access your encrypted files

Email

alice@example.com

Password

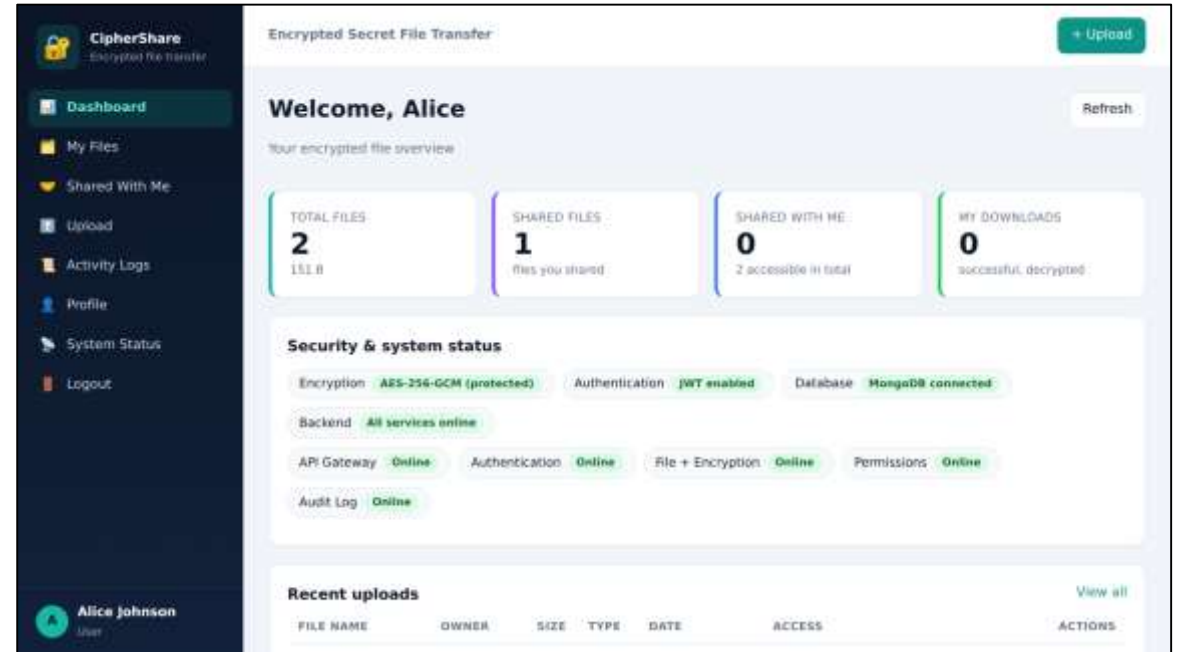
Sign in

Demo accounts (after `npm run reset:data`)

- alice@example.com
- bob@example.com
- admin@example.com

New here? [Create an account](#)

User dashboard



The user dashboard for Alice Johnson shows a sidebar with navigation links (Dashboard, My Files, Shared With Me, Upload, Activity Logs, Profile, System Status, Logout) and a main content area. The main area includes a 'Welcome, Alice' message, a 'Refresh' button, and a 'Your encrypted file overview' section with four cards: Total Files (2), Shared Files (1), Shared With Me (0), and My Downloads (0). Below this is a 'Security & system status' section with various service status indicators (Encryption, Authentication, Database, Backend, API Gateway, Audit Log, File + Encryption, Permissions) all showing as 'Online' or 'protected'. At the bottom, there is a 'Recent uploads' table with columns for File Name, Owner, Size, Type, Date, Access, and Actions.

CipherShare
Encrypted file transfer

Encrypted Secret File Transfer [+ Upload](#)

Welcome, Alice [Refresh](#)

Your encrypted file overview

Metric	Value	Description
TOTAL FILES	2	131 B
SHARED FILES	1	files you shared
SHARED WITH ME	0	2 accessible in total
MY DOWNLOADS	0	successful, decrypted

Security & system status

- Encryption: AES-256-GCM (protected)
- Authentication: JWT enabled
- Database: MongoDB connected
- Backend: All services online
- API Gateway: Online
- Authentication: Online
- File + Encryption: Online
- Permissions: Online
- Audit Log: Online

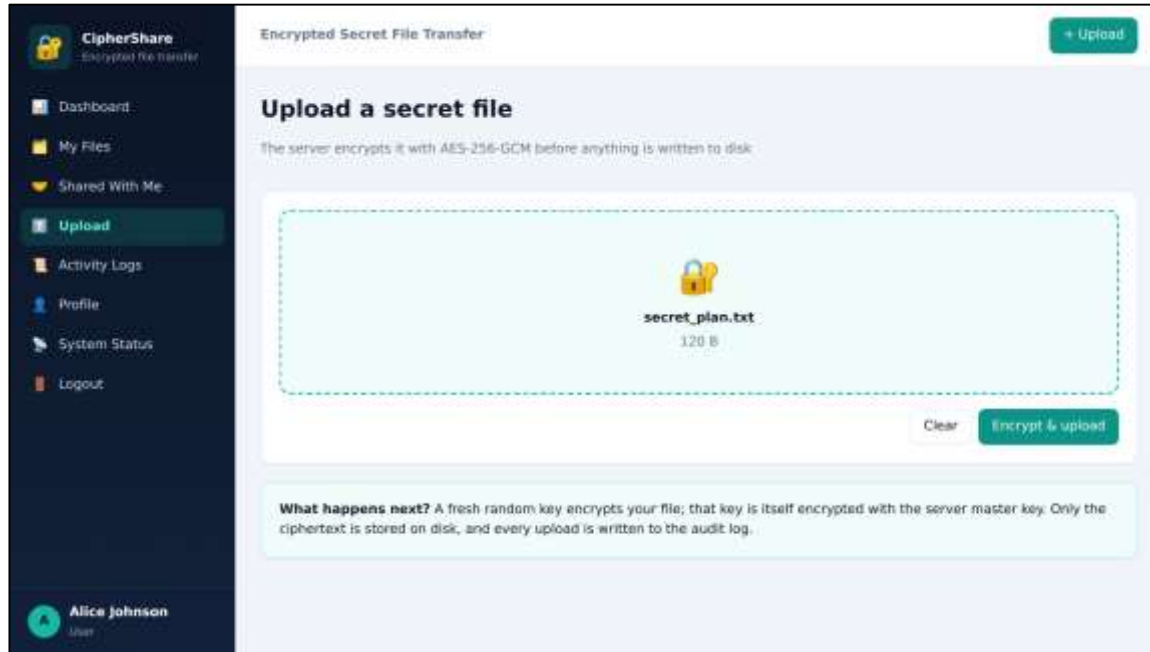
Recent uploads [View all](#)

FILE NAME	OWNER	SIZE	TYPE	DATE	ACCESS	ACTIONS
-----------	-------	------	------	------	--------	---------

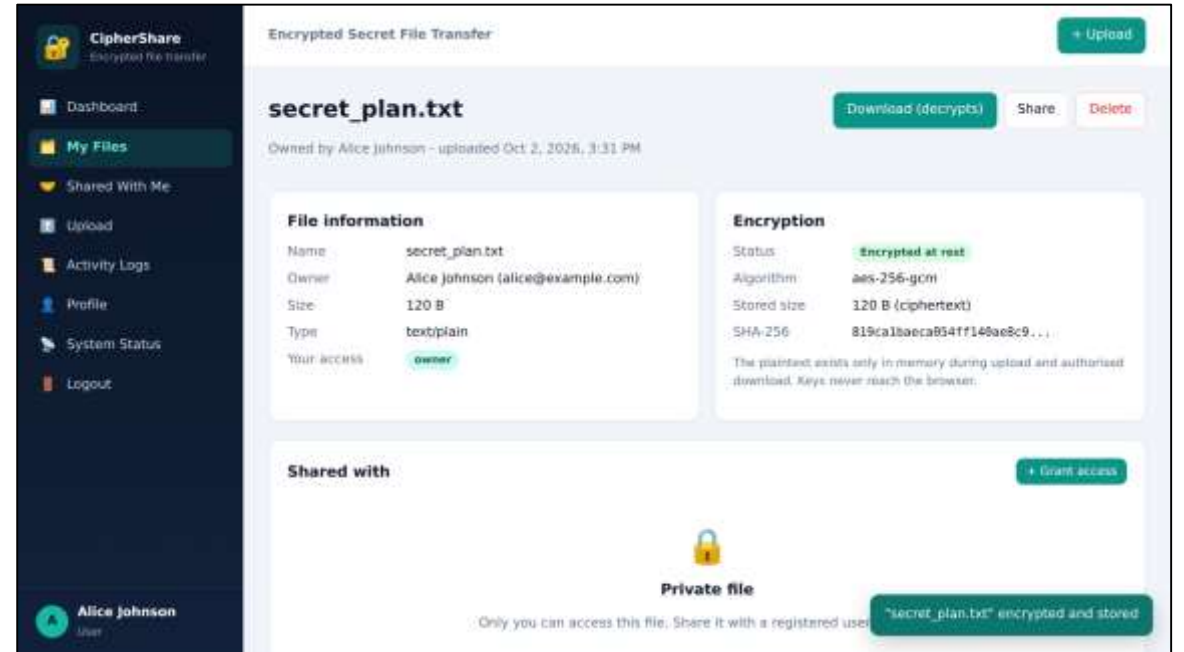
- Users sign in with email and password; a JWT is issued and used for every API call.
- The dashboard shows file counts, shared files and the live status of every backend service.

OUTPUT: UPLOAD AND ENCRYPTION

Upload page: file selected



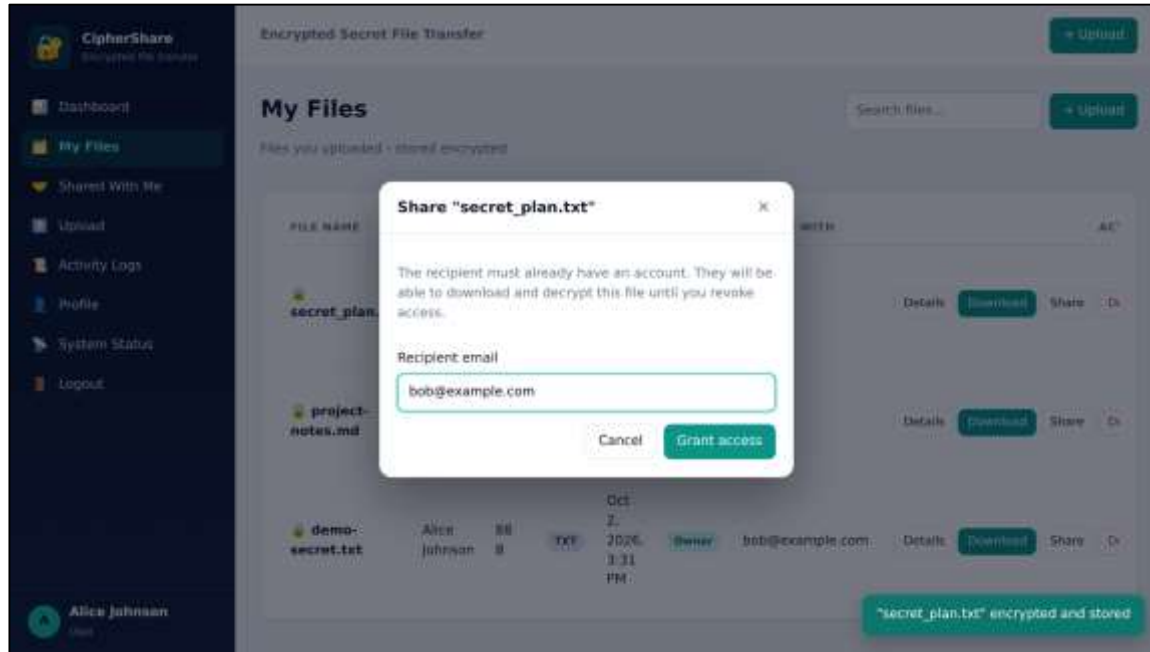
File details: encrypted at rest



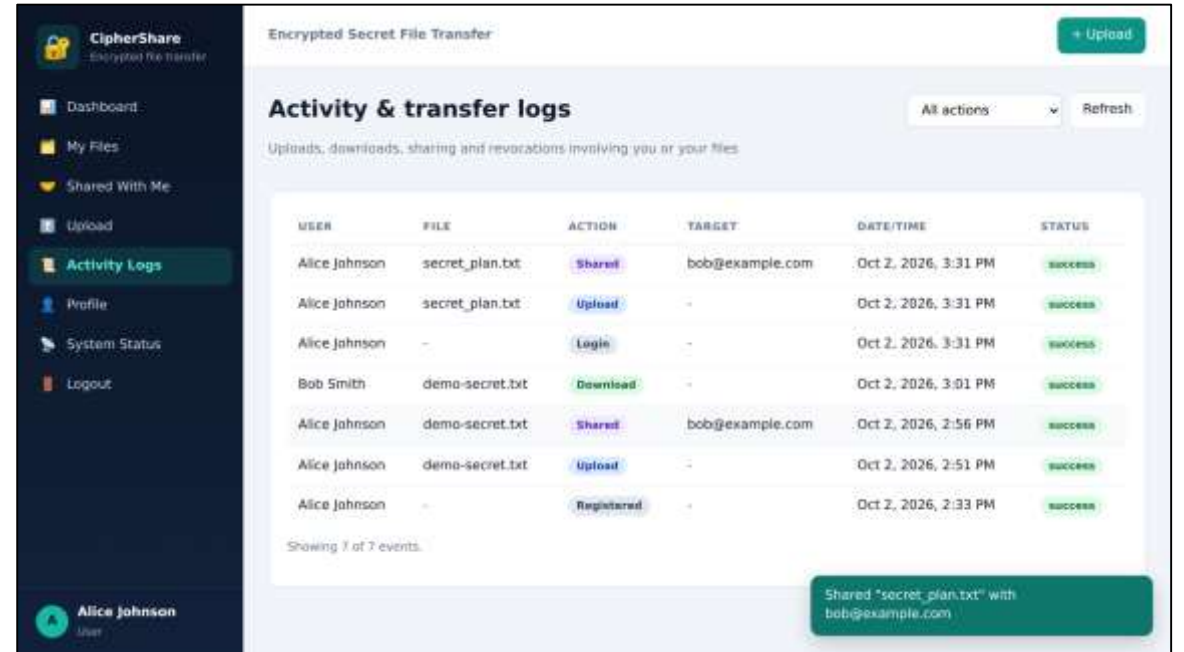
- The file is encrypted with AES-256-GCM on the server before it is stored.
- Details show the algorithm, ciphertext size and SHA-256 hash; keys never reach the browser.

OUTPUT: SHARING AND ACTIVITY LOGS

Share dialog: grant access by email



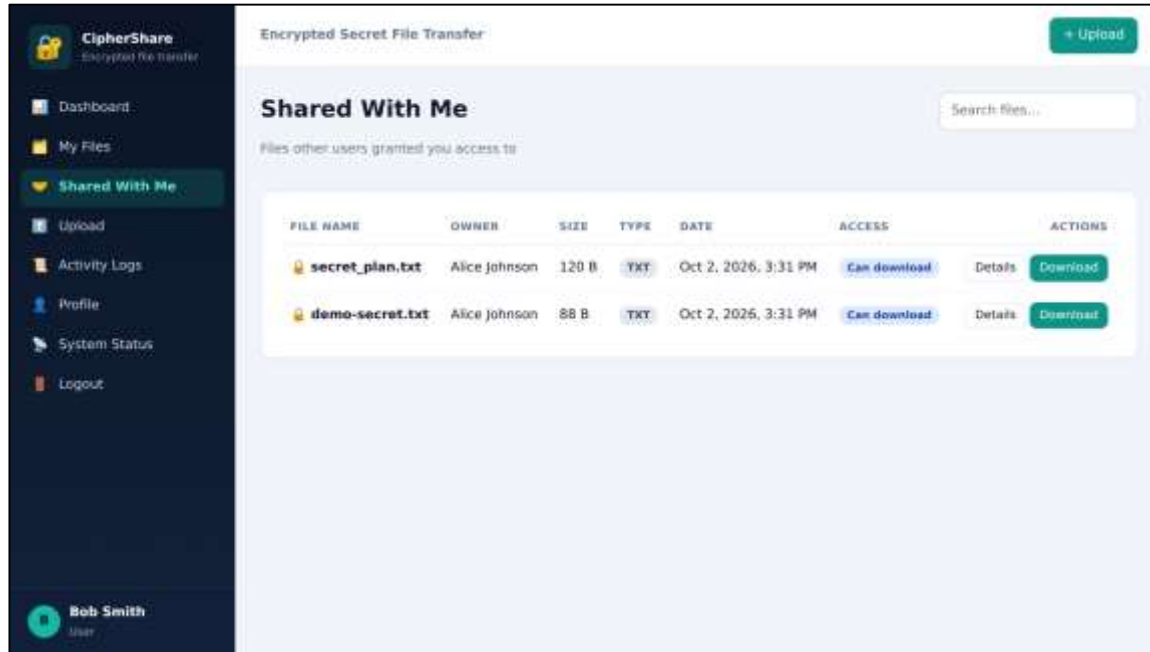
Activity and transfer logs



- The owner shares a file with a registered user by entering the recipient email.
- Every upload, share, download and login is recorded in the audit log.

OUTPUT: ACCESS CONTROL

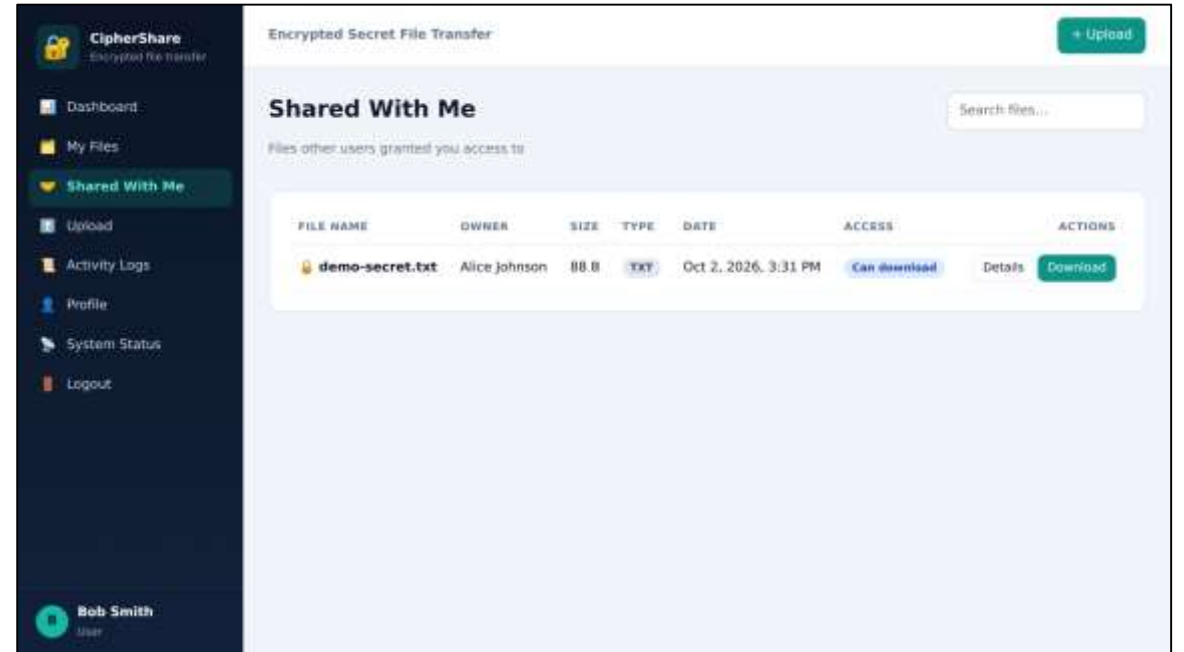
Recipient: Shared With Me



The screenshot shows the CipherShare web interface. The left sidebar contains navigation links: Dashboard, My Files, Shared With Me (highlighted), Upload, Activity Logs, Profile, System Status, and Logout. The main content area is titled 'Encrypted Secret File Transfer' and 'Shared With Me'. It displays a table of files shared with the user.

FILE NAME	OWNER	SIZE	TYPE	DATE	ACCESS	ACTIONS
secret_plan.txt	Alice Johnson	120 B	TXT	Oct 2, 2026, 3:31 PM	Can download	Details Download
demo-secret.txt	Alice Johnson	88 B	TXT	Oct 2, 2026, 3:31 PM	Can download	Details Download

Recipient after owner revokes access



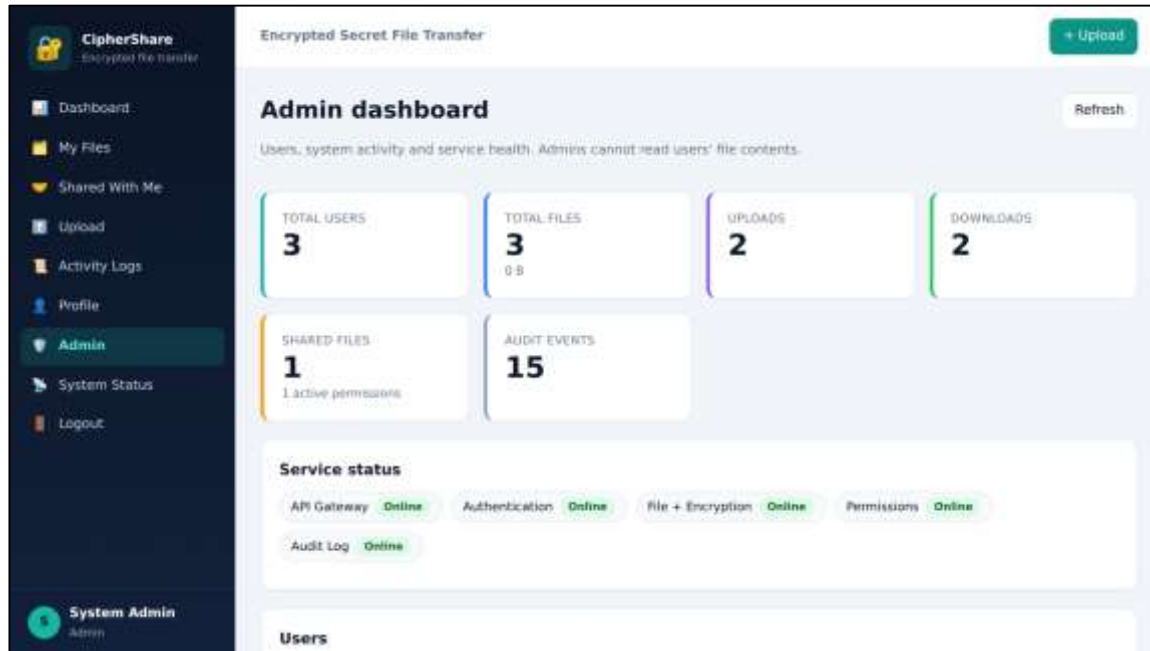
The screenshot shows the same CipherShare web interface, but the file 'secret_plan.txt' has been removed from the 'Shared With Me' list, leaving only 'demo-secret.txt'.

FILE NAME	OWNER	SIZE	TYPE	DATE	ACCESS	ACTIONS
demo-secret.txt	Alice Johnson	88 B	TXT	Oct 2, 2026, 3:31 PM	Can download	Details Download

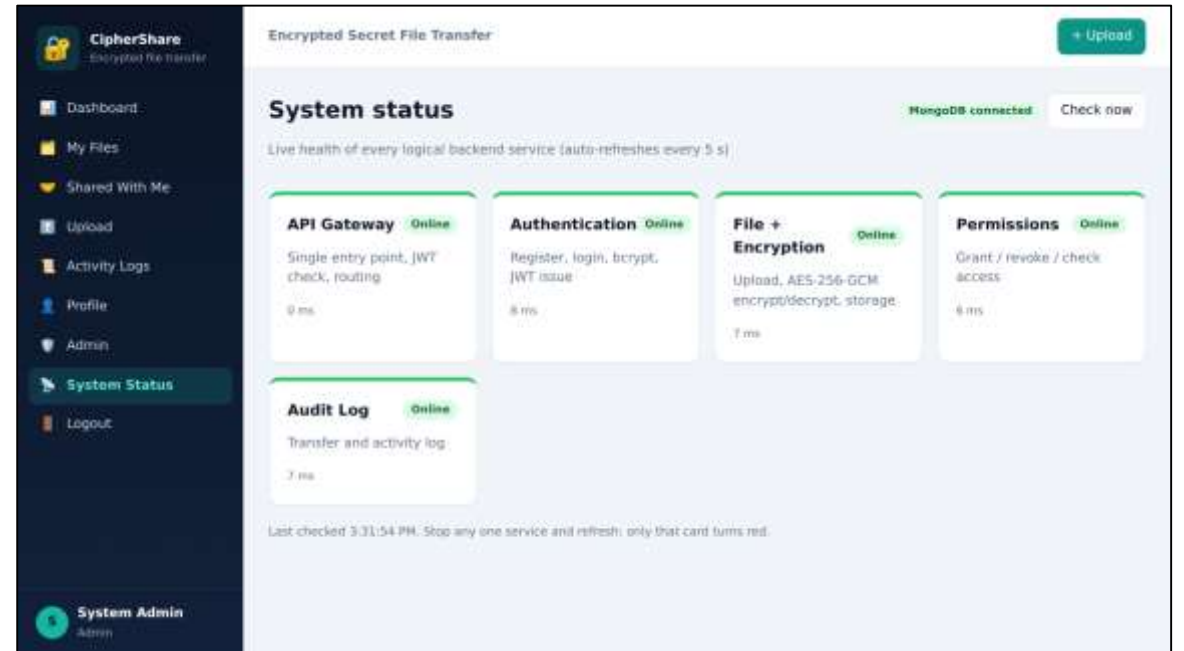
- The recipient can download only files shared with them.
- After the owner revokes access, the file disappears and further downloads are denied (403).

OUTPUT: ADMIN AND SYSTEM STATUS

Admin dashboard



System status page



- Administrators see users, totals and system-wide logs, but cannot read other users' files.
- Each backend service reports its own health, so one failing service is easy to isolate.

OUTPUT: ENCRYPTED STORAGE

Stored file on disk: ciphertext only

```
$ ls -l uploads/encrypted/
-rw----- 56 bytes 74bee6c48c3abd24819a0b01fe807c2f.enc
-rw----- 88 bytes b6fc0eaa1f0467d0ec0651ee85440387.enc
-rw----- 63 bytes d5b396770f8161302afc76e797e7082c.enc

$ hexdump -C uploads/encrypted/b6fc0eaa...0387.enc
00000000 f2 7d 07 4d fd 04 37 fb 19 8f 2c ec 3d ef 30 98 |.}.M..7....,=.0.|
00000010 41 86 28 a2 12 c4 eb 86 30 37 a8 2e c9 7b d9 4b |A.{.....07...{.K|
00000020 4c 61 e6 21 f0 99 47 a3 16 18 fd 2f 18 f0 18 92 |La.!...G..../....|
00000030 c6 18 f3 9b f0 58 fd d1 f8 b7 3d ab 07 62 16 6a |.....X....=..b.j|
00000040 c4 cd bf 24 c8 34 6a dd 62 89 2d c2 dc 7e 0c 53 |...$.4j.b....~.S|
00000050 fb 1d 2c ce b6 ca e0 5a |...,...Z|
00000058

# only ciphertext is stored on disk - no readable text
```

MongoDB record: metadata and wrapped key

```
> db.files.findOne({ originalName: "demo-secret.txt" })
{
  "originalName": "demo-secret.txt",
  "mimeType": "text/plain",
  "size": 88,
  "encryptedSize": 88,
  "sha256": "1fea8334620d2ec59b9a62a8...",
  "storedName": "b6fc0eaa1f0467d0ec0651ee85440387.enc",
  "encryption": {
    "algorithm": "aes-256-gcm",
    "keyVersion": 1,
    "iv": "sg5k1/ElzEAl9oJj",
    "authTag": "Dfhk+oqbC9QpXLEQbC3g1A==",
    "wrappedKey": "N4yZx9GKUWc9Fa9austNBGQ4...",
    "wrapIv": "RJy1eX947GMcAy/g",
    "wrapTag": "ZKl4ozGTD2/mBFvXYisNvA=="
  }
}
```

OUTPUT: TESTING AND VERIFICATION

Access denied (403 / 401), unit tests, syntax check

```
$ curl -i /api/files/<id>/download (Bob, file NOT shared with him)
HTTP/1.1 403 Forbidden
{"error":"You do not have permission to access this file"}

$ curl -i /api/files/<id>/download (no token)
HTTP/1.1 401 Unauthorized
{"error":"Authentication required"}

$ curl /api/files/<id>/download (Alice, owner)
HTTP 200 -> 63 bytes of decrypted content

$ npm run test:unit
ok 1 - encrypt -> decrypt round trip
ok 2 - every file gets a unique key and IV
ok 3 - tampered ciphertext is rejected (GCM auth tag)
ok 4 - wrong master key cannot decrypt
ok 5 - 64-hex master key is used raw
ok 6 - storage rejects path traversal and non-generated names
ok 7 - validation helpers
# tests 7
# pass 7
# fail 0

$ npm run check
OK - 38 files passed syntax check
```

End-to-end smoke test: 25 checks passed

```
$ npm run test:smoke
Running smoke tests against http://localhost:4000
ok 1. gateway is running
ok 2. auth service reachable + health endpoint
ok 13. service status lists all services
ok 3. registration
ok 4. login returns JWT
ok wrong password rejected
ok 5. JWT authenticates /api/auth/me
ok missing token -> 401
ok invalid token -> 401
ok 6. file upload
ok 7. file metadata created
ok metadata hides keys/paths
ok stored file is encrypted on disk (plaintext marker absent)
ok 11. unauthorized download rejected (403)
ok 8. file sharing
ok 9. non-owner cannot share
ok file appears in "Shared With Me"
ok 10. authorized download returns original content
ok revoke access
ok download denied after revoke
ok 12. audit logs (UPLOAD/SHARE/DOWNLOAD/REVOKE/DENIED)
ok normal user blocked from admin API
ok admin stats
ok admin cannot download others' files
ok dashboard overview

Smoke test passed (25 checks).
```

CONCLUSION AND FUTURE SCOPE

The Encrypted Secret File Transfer Distributed Backend Database System was implemented and tested end to end. By combining AES-256-GCM encryption, bcrypt and JWT authentication, per-file access control, MongoDB storage, and independent backend services, it protects confidential files and keeps a full audit trail of all activities.

In the future, the system can be extended with end-to-end encryption in the browser, key rotation with a managed key service, multi-factor authentication, HTTPS/TLS, cloud object storage, file-expiration controls, and deployment of each service on its own host for better scalability.

THANK YOU